Tracing Technical Debt by AI-Based Correlation Agent — A Case Study of Nokia Corporation

Satu Grönroos, Tuula Hietanen, Anni Salmi, Katja Turtiainen

2025 Laurea

Laurea University of Applied Sciences

Tracing Technical Debt by AI-Based Correlation Agent — A Case Study of Nokia Corporation

Satu Grönroos
Tuula Hietanen
Anni Salmi
Katja Turtiainen
Business Information Technology
Thesis
November 2025

Laurea University of Applied Sciences **Abstract**
Business Information Technology

Satu Grönroos, Tuula Hietanen, Anni Salmi, Katja Turtiainen
Tracing Technical Debt by AI-Based Correlation Agent
— A Case Study of Nokia Corporation
Year 2025 Number of pages 46

The objective of the project was to design and evaluate a solution that improves the efficiency of software maintenance processes and reduces technical debt within development environments. Nokia provided the broad development challenge on the theme of Artificial Intelligence, thereby serving as the commissioning organization for the project's overall scope. The project team then specified the purpose to create and assess a concept for an AI-powered Correlation Agent capable of identifying root causes of technical debt and supporting proactive maintenance.

The development task was to conceptualize and prototype an AI agent that integrates structural code data with dispersed textual contexts such as Git commit messages and documentation, in order to diagnose and mitigate technical debt. Nokia benefits from reduced maintenance costs, faster workflows, and improved code quality based on the potential of this solution.

The study builds on theories of technical debt management, agentic AI versus generative AI capabilities, and correlation analysis as a statistical method for identifying relationships between key indicators and maintenance goals. The research employed a Design Sprint approach complemented by brainstorming sessions, current-state mapping, benchmarking of existing tools (CodeScene, SonarQube, Greptile, Powerdrill), and structured questionnaire interviews with IT Systems Specialists.

The tools that are already available do not combine code analysis with contextual information, which makes it more difficult to solve problems. Agentic AI, due to its inherent multi-step reasoning capabilities, is highly suitable for multi-step maintenance workflows and proactive risk detection. A prototype named Code Colleague (CoCo) demonstrated that a conversational interface enhances collaboration between developers and the AI agent.

The Correlation AI Agent can significantly reduce manual investigation time, accelerate fault correction, and lower long-term maintenance costs. The focus of future development should be on integrating the Correlation AI Agent into a multi-agent chain, defining inter-agent collaboration protocols, and establishing auditing mechanisms for AI-driven processes.

Keywords: Technical Debt, Agentic AI, Correlation AI Agent, Software Maintenance, Design Sprint

Contents

1 Introduction

The thesis was implemented as a Design Sprint Challenge at Laurea University of Applied Sciences in collaboration with Nokia Corporation as the client company. The Nokia representatives presented the key concepts for the project and two Design Sprint challenges. This project is a proposed solution to the challenge “How can Generative and Agentic AI transform the way we design, build, and release software and hardware products?”.

The primary goal of the development work is to directly address and reduce the drain on business operations by mitigating the effects of technical debt. This study proposes an Agentic Artificial Intelligence Platform designed to fundamentally transform code health management. This solution moves past traditional, isolated code scanners by creating an intelligent layer that unifies disparate data silos, both technical and text-based, to proactively detect, categorize, and report on debt. By making the maintenance workflow easier and more accurate, the platform will reduce code maintenance costs and free up developer time for higher-value projects.

Technical debt is the hidden cost that emerges when development is constrained by time or budget, forcing teams to prioritize speedy delivery over long-term architectural perfection. It represents the accumulated expense incurred when strategic trade-offs are made or when schedule pressures lead to suboptimal architectures, complex code, or neglected documentation. This debt extracts a heavy toll on organizations. Financially, it manifests as maintenance costs, reduced reliability, and a significantly slower time-to-market for new features. For developers, technical debt hinders workflow, forcing valuable time to be spent navigating, patching, and maintaining codebases rather than building new products. The research analysis confirms that this burden directly translates into substantial business expenses that limit innovation. (Friedland B., De Santis M. 2025, 3—5.)

The data framework for this solution is built on two core components. First, the methodology involves mapping existing code maintenance systems, protocols, and error-detection tools. Second, crucial, complementary data sources are identified that are currently siloed and disconnected from the codebase analysis. These include Git commit

messages, internal code comments (even from original developers who may no longer be with the company), design documents, and original requirement documents. The core of the solution is the Correlation AI Agent, which is tasked with the initiative role of fusing these technical and text-based silos to detect errors and risk factors, establish traceability, report inconsistencies, and suggest solutions, thereby developing sustainable practices for codebase maintenance.

The Correlation AI Agent is defined by its proactive nature and structured interaction model. Its capabilities cover code structure analysis, documentation completeness checks, and contextual log evaluation. However, the system is fundamentally built around a Human-in-the-Loop idea to ensure continuous monitoring of the error detection and fixing process. The Correlation AI Agent acts as an intelligent assistant, constantly assessing the situation of the codebase and its compliance with requirements. It uses specific touchpoints to communicate findings to the human user, presenting a summary of its proposed solutions. The user then validates or corrects the insight, allowing the AI agent to learn and proceed. This cooperative structure ensures that the final decisions are informed by data but remain under human control.

The team worked during a five-day Design Sprint week to develop the proposed solution. Several research methods were used for ideating, understanding, defining, conceptualization, and decision-making phases of the development process. Based on the knowledge gathered, the team had brainstorming sessions, mapped out current state challenges, and benchmarked potential competing solutions. Three interviews were performed to get context for creating potential use case scenarios. A prototype was created to demonstrate the use case scenarios and to gather feedback on the proposed solution.

The project's background is introduced, followed by the knowledge base on solving technical debt with artificial intelligence. The research methods used during the process are described. For further specification, the Human-in-the-Loop code repair and maintenance workflow with the Correlation AI Agent is explained. The prototype that the team built is described. Finally, the team's conclusions and future scope are reported.

In this report, artificial intelligence has been used in source searching, language design, and creating diagram figures.

2 Project's Background

The project was done in collaboration with Nokia Corporation, which provided the challenge the team worked on. The process started with a meeting in which Nokia representatives introduced Nokia Corporation and the concepts of Network as Code and Agentic AI, followed by case examples. Subsequently, the design challenges for the project were presented. After conducting research, the team chose the challenge regarding AI.

2.1 Nokia Corporation

Nokia Corporation is a globally operating Finnish company that creates technology to help the world act together. By utilizing their work across mobile, fixed, and cloud networks, they are leading the way in B2B technology innovation, creating networks that sense, think, and act. (Nokia 2025a; Nokia 2025b.) Nokia is a trusted partner of Laurea University of Applied Sciences in the field of critical networks. With this partnership, Nokia and Laurea can establish structured and systematic cooperation, carry out collaborative projects with students, and offer students a view of future job opportunities at Nokia. The collaboration's aim is to create new opportunities for both parties. (Laurea 2022.)

Nokia announced on February 27, 2025, that it was implementing new agentic AI capabilities to its portfolio of autonomous networks to help their Communication Service Providers (CSPs) better automate, secure, and monetize their networks. According to Andy Hicks, Senior Principal Analyst at GlobalData, traditional machine learning, LLMs, and agentic AI will all be essential to the transition to completely autonomous networks. (Nokia 2025c.)

2.2 Project Implementation and Stakeholder Collaboration

The project was implemented as a team. A team of four members worked together during the entire process. The proposed solution was developed, keeping in mind the original design challenge and the needs that Nokia could have regarding AI in research and development.

During the process, there were two sparring sessions with Nokia representatives to gather important insights on the project. The feedback was used to refine the scope and the team's proposed solution. The final proposed solution and the prototype were pitched to

Nokia representatives to gather insights. The project was further refined based on the commentary.

3 Solving Technical Debt with Artificial Intelligence

In the project, the focus was on how artificial intelligence can be used to solve the challenges of technical debt. In a sparring session with Nokia, the team received guidelines regarding their company's potential needs. An important aspect of gathering the knowledge base was to search for information on technical debt and agentic and generative artificial intelligence.

3.1 Technical Debt in Software Development

Technical debt refers to the long-term costs and maintenance challenges that arise from short-term compromises made during software development. The term was first introduced by Ward Cunningham in 1992, who likened it to financial debt: quick delivery may be beneficial in the short term, but it requires "repayment" through future refactoring and code improvement efforts. (Cunningham 1992.)

Martin Fowler expanded on Cunningham's concept by introducing the *Technical Debt Quadrant*, which categorizes debt based on two dimensions: reckless vs. prudent and deliberate vs. inadvertent (Fowler 2009). This framework helps teams understand whether the debt was a conscious trade-off or the result of poor decision-making.

Beyond this classification, technical debt manifests in several distinct forms. Architectural debt arises from foundational system flaws that hinder scalability, flexibility, or maintainability. Code Debt results from inconsistent coding practices, poor documentation, and rushed development. Infrastructure and DevOps Debt emerges from outdated deployment pipelines and inefficient automation processes. Process Debt caused by unclear workflows, poor collaboration, and a lack of agile practices. Security Debt occurs when encryption, authentication, and vulnerability management are neglected, increasing exposure to cyber threats. (Mucci 2025.)

Like financial debt, technical debt accrues interest over time. If left unmanaged, it leads to increased maintenance costs: developers spend more time fixing bugs than building new features. On average, developers spend one-third of their time addressing technical

debt. (Raghuvanshi 2024.) Technical debt leading to reduced competitiveness, slower innovation, and degraded performance can result in customer churn and reputational damage. Furthermore, unaddressed security vulnerabilities and regulatory risks may result in compliance violations and serious legal consequences. (Mucci 2025.) Effective management of technical debt requires balancing speed, quality, and cost. Organizations must make strategic decisions about when to incur debt and when to invest in its repayment. (Mucci 2025.)

Generative AI tools can accelerate development by automating repetitive tasks and improving code quality. They help identify redundant code, enhance readability, and generate high-quality boilerplate code. However, without human oversight, AI-generated code may introduce inconsistencies or unnecessary dependencies, increasing long-term debt. (Mucci 2025.)

Project management and automation tools, such as SonarQube and DebtFlag, support technical debt tracking and prioritization. These tools help identify bottlenecks, monitor code quality, and ensure refactoring tasks are addressed in the backlog. (Mucci 2025.)

Technical debt is not solely a technical issue; it is also cultural. Organizations that promote proper documentation, maintainable APIs, and long-term software health are better equipped to prevent debt accumulation. Low-code and no-code platforms reduce manual coding errors and streamline development, helping minimize the risk of technical debt. Technical debt is an inherent aspect of software development. Its management requires strategic thinking, technical expertise, and cultural commitment. When handled effectively, technical debt can serve as a tool for achieving business goals. However, if neglected, it threatens system sustainability and organizational agility. (Mucci 2025.)

3.2 Agentic AI

An advanced form of artificial intelligence, known as agentic AI, can plan, execute, and optimize complex business processes independently by combining autonomous decision-making and advanced reasoning ability. Through fusing more advanced features, such as huge language models and more traditional machine learning, agentic AI can create AI agents that can analyze data, set goals, and take actions with minimal human intervention. (Sawant 2025.)

One of the most critical features of agentic AI is its autonomy, particularly in the context of complex multi-goal scenarios. Agentic AI systems can transition from one basic task to multiple complicated end goals and progress through several necessary activities. (Acharya, Divya & Kuppan 2025.)

Agentic AI can function in a variety of changing environments and incorporate real-world variability. It can swiftly adapt to changes in environment, data, patterns, or freshly formed user requirements. This is made possible by the fact that the agentic AI system can interact with its surroundings, process data in real-time, and understand situational context. These capabilities enable the system to monitor and actively participate in operational parameters that are subject to change, even at the last minute. (Acharya, Divya & Kuppan 2025.)

Agentic AI can work independently for extended periods of time because of its flexibility and autonomy. With inherent flexibility and autonomy, agentic AI can work independently for long durations. It refines its behavior and decision-making by learning from its context over time, a process that is rapidly reinforced through continuous human feedback. With flexibility, agentic AI can pursue the appropriate objectives and behave differently in the same situation. Goals can be prioritized, potential actions and their results can be evaluated, strategies can be rethought, and adjustments can be made in response to new information in a changing environment. (Acharya, Divya & Kuppan 2025.)

3.2.1 Agentic AI vs. Generative AI

Artificial intelligence that can produce original content when prompted by a human is known as generative AI. Text, images, videos, audio, or software code are a few examples of original content in question. It is based on machine learning models known as deep learning models, which are algorithms that mimic how the human brain learns and makes decisions. Generative AI models are made to recognize and encode patterns and relationships in vast amounts of data. They can then use this information to comprehend natural language requests from humans and immediately produce original content based on the data they were trained on. (Downie & Finn 2025.)

Generative and agentic AI have both similar and different use cases. Businesses utilize generative AI to create vast amounts of search engine-optimized content, such as blogs and landing pages, which aid in generating organic traffic. Chatbots and virtual assistants

are used by sales teams to identify and develop leads. To speed up the product development cycle, companies have also found use for generative AI in developing new product concepts or designs based on consumer preferences, market data, and trends. Additionally, generative AI can be used to generate automated solutions to customer service queries, such as answers to frequently asked questions and real-time problem-solving. (Downie & Finn 2025.)

Like generative AI, agentic AI is also utilized in customer service, but in a more advanced way, because traditional customer service chatbot models had limitations because of the technology's pre-programmed nature and occasionally needed human intervention. However, because autonomous agents can predict a situation and help, they are able to determine a customer's intent and emotion immediately and take action to address the problem. They can collect, clean, and prepare data for an organization, which allows them to automate operations. (Downie & Finn 2025.)

Since its cybersecurity features are helpful for patient data and privacy issues, agentic AI is also used in the healthcare industry for diagnostics, patient care, and streamlining administrative duties. Because it can automate internal workflows to make things simpler for human employees without requiring their physical presence, it is also used to manage company processes autonomously and perform difficult jobs like reordering inventory and managing supply chain operations. Additionally, by evaluating market trends and financial data to make independent judgments on investments and credit risks, it can assist industries in meeting customer objectives and optimizing outcomes in real-time. (Downie & Finn 2025.)

3.2.2 Code Maintenance

Based on the capabilities outlined in the literature review, agentic AI is highly suitable for assisting in code maintenance. The research demonstrates that agentic AI possesses the capacity to autonomously manage complex business processes and automate internal workflows, such as handling supply chains and inventory reordering.

The ability of agentic AI translates directly to the potential for conducting the structured, multi-step process of code maintenance. It is essential to note that in this role, agentic AI acts as the process conductor and decision-maker. It utilizes generative AI technology as a tool for executing specific steps within its workflow, such as generating the actual code and solution proposals.

3.3 Correlation Analysis

Correlation analysis is a statistical method in which the strength and direction of relationships between quantitative variables are assessed. Correlation is the linear connection of two continuous variables. It can be positive, negative, or non-existent. If two or more variables have a high correlation, it means that they have a strong relationship. Having a weak relationship indicates that the variables are scarcely related to each other. (Franzese & Antonella 2018; Kestilä-Kekkonen 2021.)

During the development of the project, correlation analysis helps the proposed Correlation AI Agent identify how factors such as technical debt indicators (e.g., code complexity or test coverage) relate to software maintenance goals. By analyzing these correlations, the Correlation AI Agent can prioritize actions that most effectively reduce maintenance effort and improve software quality.

4 Research Methods

The development process took place during Design Sprint week. The team utilized several research methods across the different phases to develop a proposed solution to the challenge. For understanding and defining the problem, the methods included brainstorming, mapping, and benchmarking. Conceptualization and subsequent decision-making relied on structured questionnaire interviews and use cases before moving into the prototyping phase.

4.1 Design Sprint

Design Sprint is a five-day methodology designed to ideate, build, and test a prototype, progressing from problem to tested solution. The process allocates a specific goal to each day. A small team works effectively to solve the problem step-by-step (Figure 1). (Knapp & Zeratsky 2025.)

The project team brainstormed to generate different ideas for the challenge that Nokia had presented. Following research into the feasibility of the initial challenge areas and the concepts generated, team members came up with several ideas, which were then discussed and reviewed how viable they could be.

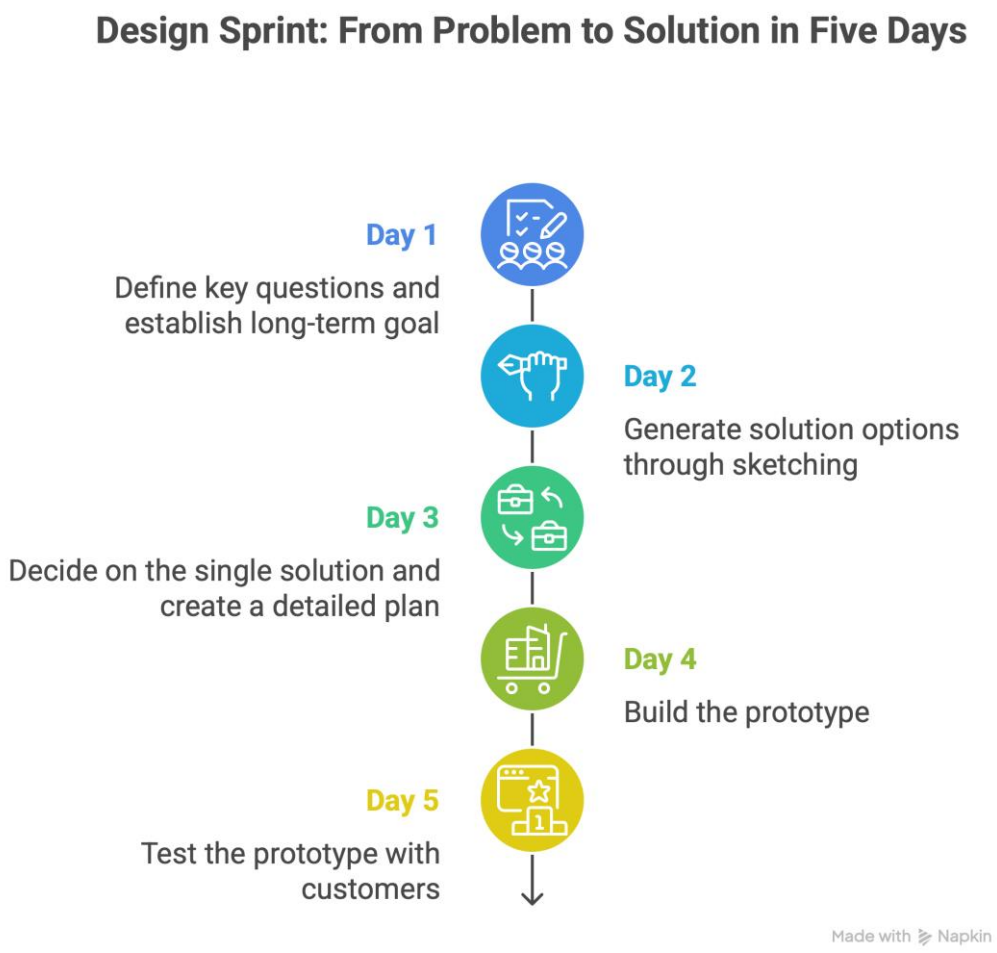


Figure 1: Design Sprint: From Problem to Solution in Five Days

On the first day of the Design Sprint week, the team defines key questions and establishes a long-term goal. Subsequently, a simple map of the product or service is created, and knowledge is shared among team members. The goal of the day is to pick a target moment from the map, one that represents the greatest risk or opportunity. The second day focuses on problem-solving, where solution options are generated through sketching rather than traditional brainstorming. The third day's objective is to decide on the single solution that will be prototyped and to create a detailed plan for its construction. On the fourth day, the team builds the prototype, which is then tested during the fifth day by showing it to actual customers to gather critical feedback (Knapp & Zeratsky 2025).

4.2 Brainstorming

Brainstorming is a creative problem-solving method that focuses on innovation. The goal is to generate new ideas. In the brainstorming process, a group of people work together on

a certain theme, generating new theme-related ideas. To maximize the number of ideas generated, the core objective is to encourage all participants to express their thoughts freely. The brainstorming process begins by warming up and trying to release all factors that limit creative thinking. Then, at the ideation phase, the ideas are generated in free form and without criticism. In the selection phase, the ideas are reviewed critically, and the most executable idea is chosen. (Ojasalo, Moilanen & Ritalahti 2015, 44, 89, 160—161.)

On the first day of the Design Sprint week, the team had a brainstorming session based on the feedback from the first sparring session with Nokia and the knowledge gathered. The outcome of the session was a mind map with the most important findings (Appendix 1). It was used to define the scope of the Correlation AI Agent, which identifies technical debt and reduces software maintenance goals.

4.3 Mapping out the Current State Challenges in Technical Debt Diagnosis

The long-term viability and efficiency of software development are fundamentally challenged by technical debt. Although numerous tools exist to identify structural code issues, the core problem in modern software maintenance lies not in detecting the debt itself, but in diagnosing and understanding the root causes of its origin.

Technical debt is unavoidable and can even be beneficial, provided it's properly managed. However, its management is challenging because it stems from various sources, has unpredictable effects, and involves assessing future risk. Managing technical debt is essentially risk management. Unmanaged debt will accumulate over time, potentially leading to a crisis. (Allman, E. 2012.)

This chapter maps out the current state of technical debt management, highlighting critical issues stemming from data fragmentation and the context blindness of existing analysis tools, which ultimately leads to inefficient developer workflows and increased maintenance costs.

4.3.1 Data Silos and Context Blindness

The primary challenge in managing technical debt is not a lack of data, but rather its fragmentation across distinct, siloed environments. Data silos are essentially isolated data storage areas that cannot easily communicate with the rest of the systems. One

reason for the isolation is technological, caused by different applications and data systems not being built to talk to each other (Tsidulko, J. 2023).

Data silos result in inefficiencies and data inconsistencies, making it harder to extract key insights. They also serve as a barrier to teamwork, communication, and innovation. Because modern success depends on data-driven decisions based on a massive volume of diverse data, organizations that fail to eliminate silos are at a severe disadvantage in using their data to secure a competitive edge. (Kumar, S. 2023.)

Therefore, a significant challenge in the software product lifecycle is that the tools used for code maintenance operate in data silos and are unable to consolidate text-based, contextual information with the structural problem, which is the code itself. Essential information is fragmented across various systems such as logs, design documents, and Git commits. Developers are forced to manually combine this information to investigate issues, which are slow, inefficient, and prone to errors.

4.3.2 Limitations of Code Analysis

Code analysis tools are divided into static and dynamic tools. Static tools scan code without executing the program, localizing syntax errors and complexity, and reporting their findings. Dynamic tools, conversely, analyze the code during execution, detecting problems that only emerge at runtime (Battacharjee, 2025).

Effective software maintenance fundamentally relies on observability. Observability is the capacity to monitor and comprehend an application's or system's internal condition using its external data outputs, such as logs, metrics, and traces (Protalinski 2023). Despite its importance, observability on its own is insufficient to handle the ever-increasing volume, velocity, and variety of data being generated (Kumar 2023).

However, these tools and methods are context blind. They see numerical code complexity but do not know if this complexity is intentional, for example, required by an external standard, or a result of poor design. Text documents like design documents and code comments are sources of human understanding and context, but they remain separate from the code. Consequently, developers spend time on manual retrieval to figure out why the code was written in a particular way.

4.3.3 Communication in Technical Debt and Workload

Unclear, outdated, or incomplete in-code documentation contributes to technical debt rooted in communication issues, which leads developers to waste time on code interpretation. When a defect is found, this inefficiency often results in an investigation period that is significantly longer than the actual fix.

The lack of completeness analysis, meaning the reliability of the overall context, forces the developer into unnecessary work. Struggling with poorly documented code can decrease developer productivity and motivation. Research establishes software documentation as a vital component of software quality. When documentation is poor, outdated, or absent, it becomes a primary cause of errors in development and maintenance. Notably, most defects exposed during integration testing stem from failures in the initial design and requirements documentation, indicating that documentation errors often precede code errors. (Chomal & Saini 2014.) Because a complete picture is missing, prioritizing technical debt resolution is often reactive and based on subjective assessment.

4.4 Defining the Research Question

The research question focuses on addressing these challenges by exploring how agentic AI can enhance software maintenance and technical debt management. The proposed solution aims to function automatically, integrating all relevant data sources to detect improvement areas in code blocks, thereby solving the identified findings in a sustainable and cost-efficient manner.

The core research question is formulated as follows:

> How can a Correlation AI Agent, designed to enhance technical debt diagnosis by combining and interpreting technical code data and dispersed textual context, improve the efficiency and speed of software maintenance operations?

4.4.1 Core of the Correlation AI Agent and Its Objectives

The core function of the Correlation AI Agent is its ability to merge technical code data and dispersed textual context to achieve a reliable and maximally complete picture of a code issue. As established, the problem is not the lack of data, but rather its fragmentation into

silos. The central idea is that by including content from non-code-based data sources in the analysis, a reliable and complete overall context is established. This capability directly speeds up and simplifies maintenance actions, which significantly reduces the time spent on issue investigation.

4.4.2 Suitability of Agentic AI

The decision to leverage agentic AI for code maintenance is strongly backed by the technology's core capabilities: decomposing complex situations, planning actions based on reasoning, and setting goals (Garg 2025). Agentic AI fits this role exceptionally well because it introduces two critical capabilities that are essential for achieving the objective of enhanced code maintenance.

First, the AI agent has the potential to combine data silos and decompose a complex problem into manageable parts before developing an action plan. Secondly, the AI agent's automated work enables early and proactive detection of technical debt. It can continuously scan the codebase for risk factors and examine them within the comprehensive context of the assigned data sources.

Based on its analysis, the AI agent could present its findings to a human, along with suggested courses of action. The long-term goal is thus to mitigate technical debt by examining risks before they escalate into more costly levels.

4.5 Benchmarking

Benchmarking is a method based on researching how other organizations operate. Usually, successful organizations are researched to discover their success factors. The other organizations' proven procedures can then be implemented in similar processes. Targets for benchmarking can be found among the given organization's other sections, other industries' organizations, competitors, or the industry's standards and statistical averages. (Ojasalo et al. 2015, 186.)

By benchmarking, some possible competing solutions were discovered: CodeScene, SonarQube, Greptile, and Powerdrill. The core differences in their concept and purpose are summarized in Table 1.

Comparison of Code Analysis Tools

Characteristic	CodeScene	SonarQube	Greptile	Powerdrill
Concept	Intelligent Technical Debt Prioritization Platform	Code Quality Control by Static Analysis	AI-Driven Code Review	AI Data Correlation & Analysis
Main Purpose	Prioritizing Refactoring & Visualizing Risk	Identifying Bugs, Vulnerabilities & Code Smells	Automating PR Reviews & Codebase Comprehension	Calculating and Interpreting Correlation Coefficients
Techinal Debt Management	Yes (CodeHealth™ Metric)	No	No	NO
AI/LLM-Powered Analysis	Yes (Automated reviews, local quality gates & risk detection)	Yes (Code Assurance)	Yes (Context & PR Reviews)	Yes (Correlation analysis & RAG)
Real-Time IDE Feedback	Yes (Code Health Feedback)	Yes (Error Identification and correction suggestion)	No	No
Codebase Graph/Mapping	Yes (CodeHealth™ Visualization & Git history analysis)	No	Yes (Code graph creation)	No (Heat Maps & Scatter Plots)
Natural Language / Conversational Interface	No	No	Yes (REST API interaction)	Yes (Dataset engagement with natural language)

Table 1: Comparison of Code Analysis Tools

The team investigated the concept, precision, usability, aim and goal, benefits, pain points, and challenges of each of the benchmarking targets. This subsequent analysis was conducted to map the current landscape of code analysis solutions, identify existing deficiencies, and utilize these findings to refine the proposed solution for more comprehensive technical debt management.

4.5.1 CodeScene

CodeScene is a software engineering intelligence tool that prioritizes refactoring, shows business impact, and visualizes high-risk technical debt. Its IDE (Integrated Development Environment) extensions offer real-time code health feedback, enabling users to develop code that is understandable, scalable, and maintainable, thereby preventing technical debt. It can also quickly identify critical debt that hinders productivity and prioritize reworking based on effect. (CodeScene 2025b.)

The technical debt in any codebase is shown by CodeScene's CodeHealth™ metric. It produces a visualization that helps with identifying the areas of the code that are healthy and good, as well as those that have a growing amount of technical debt. (CodeScene 2025c.)

To prevent technical debt, CodeScene uses AI-powered refactoring in editors to provide automated reviews, local quality gates, and early risk detection. (CodeScene 2025a.) It also mines Git history to identify development patterns, flags risks, and provides the insights required to scale software and teams. (CodeScene 2025b.)

By exploring Git repositories and examining trends in the development activity, CodeScene employs its Hotspot™ Analysis to assess how frequently various code sections are being worked on. Users may efficiently prioritize refactoring efforts by identifying the most intricate and often modified sections of the source. Additionally, CodeScene's CodeHealth™ Monitor provides immediate suggestions to keep the code manageable and detects declines in code health in real time. (CodeScene 2025c.)

CodeScene focuses on managing technical debt and prioritizing refactoring by providing real-time feedback on code health and visualizations of high-risk areas. (CodeScene 2025a.) Its AI-driven tools and Git history analysis help identify development trends and critical code components that impact productivity. (CodeScene 2025b.) The main challenge is ensuring that developers effectively utilize these features in daily work and that identified technical debt leads to actionable steps. Additionally, continuous code changes and complexity can make refactoring prioritization difficult, even with tool support.

4.5.2 SonarQube

SonarQube is a code quality analysis tool that uses source code analysis to find bugs, vulnerabilities, and code smells in the code. It can provide a thorough analysis report of the code inside the SonarQube server and scan the entire code for bugs, duplicates, security flaws, security hotspots, and code smells. (GeeksforGeks 2025.)

SonarQube can identify and fix problems in real time as the user writes code. When connected with SonarQube, it follows the user's IDE coding policies. It supports cloud-based and artificial intelligence IDEs, such as GitHub Codespaces, GitPod, Windsurf, Trae, and Cursor, and interfaces with several code editors, including Visual Studio Code

and IntelliJ. (Sonar 2025a.) It offers real-time analysis and immediate feedback. It identifies the code errors and offers ways to correct them. (Sonar 2025b.)

SonarQube is deployed either on-prem or in the cloud. Bitbucket Pipelines, Azure DevOps, GitLab CI/CD, and GitHub Actions can all be integrated with it to automate code reviews and display the health status of the code wherever the user works. (Sonar 2025a.)

With extensive static analysis and real-time feedback, SonarQube aims to identify and assess problems with the code early in the development process, with the goal of integrating effortlessly with the user's current workflow. (Sonar 2025c.)

By running automatic code reviews and implementing strong quality checks to proactively identify issues in AI-generated code, SonarQube's AI Code Assurance enables users to employ AI coding tools with confidence. Before going into production, projects containing AI code must pass the AI Code Assurance process to verify that every piece of the code fulfils the highest standards of quality and security requirements. (Sonar 2025a.)

It is a flexible and performant tool. Deployment options include on-premises, cloud, server, Docker, and Kubernetes. The best performance is supposed to be achieved by language-specific loading, numerous computer engines, and multi-threading. SonarQube Server enables users to concentrate on creating better and quicker code by automatically performing code quality and security checks and offering actionable code information. (Sonar 2025a.)

Pain points include managing the growing volume of AI-generated code, ensuring consistency with coding standards, and avoiding hidden security flaws. Challenges involve integrating SonarQube effectively into diverse workflows, encouraging developers to act on feedback, and maintaining rigorous quality checks across both human-written and AI-generated code. (Sonar 2025a; Sonar 2025b; Sonar 2025c.)

4.5.3 Greptile

Greptile is an artificial intelligence tool that creates a comprehensive code graph showing the connections between every function, class, file, and directory in the user's entire codebase. This understanding enables it to diagnose sentry alarms and failed tests, review pull requests, create ticket descriptions, and interact with any user tools using a straightforward REST API. (Gupta 2024.)

Greptile learns the team's coding standards and applies them to the user's PRs as it learns from the user and builds custom context. With the use of custom rules, style guidelines, and context, its behavior may be customized. Custom rules, style manuals, or documents, and miscellaneous information can all be considered custom context. (Greptile n.d.)

Greptile is used by over a thousand software teams to ship quickly. It can be used in GitHub and GitLab to automatically review PRs with the complete context of the codebase. Python, JavaScript, TypeScript, Go, Elixir, Java, C, C++, C#, Swift, PHP, and Rust are among the programming languages that Greptile supports. With a somewhat slower response, most of the other programming languages are also supported. (Greptile 2025.)

Greptile aims to have the full codebase context, catch three times as many bugs, and merge PRs four times quicker. It can comprehend how everything ties together and provide a comprehensive codebase graph. (Greptile 2025.)

4.5.4 Powerdrill

Powerdrill is an AI-powered data analysis tool that makes it easier to calculate and interpret correlation coefficients. These values indicate the direction and the strength of a linear relationship between two variables, ranging from -1 (perfect negative correlation), 0 (no correlation), and +1 (perfect positive correlation). They are used as a numerical summary, helping to support data-driven choices, validate hypotheses, and identify trends. (QQ 2025.)

As a software-as-a-service (SaaS) AI solution focused on both personal and business datasets, Powerdrill states that it makes it possible to engage with datasets for tasks using natural language. Its key competitive features include multi-modal support for multimedia input and output, accurate user intention understanding, hybrid use of large-scale high-performance Retrieval Augmented Generation (RAG) frameworks, thorough dataset comprehension through indexing, and effective code generation for data analysis. (Powerdrill n.d.)

Powerdrill uses a combination of a conversational interface and intelligence to perform correlation analysis. Some of the supported data set formats include Excel, TSV, and CSV files. It can offer a summary and compute the correlation coefficients instantaneously.

Additionally, it can provide data reports and visuals like heat maps and scatter plots. (QQ 2025.)

The purpose of Powerdrill is to make the calculation and the interpretation of correlation coefficients easier. It is a program intended to help anybody understand correlation analysis, even those without coding skills or a deep understanding of statistics. (Powerdrill n.d.)

4.6 Structured Questionnaire Interview

Interviewing is a suitable method for gathering qualitative data about people's opinions and experiences (Hakala 2024, para. Haastattelu). The goal of interviewing is to gain deeper knowledge and clarification about the subject. Interviews can give deep information about the area of development at a quick pace. (Ojasalo et al. 2015, 106.) The most structured form of interviewing, the questionnaire interview, is usually used when a quick solution is needed (Hakala 2024, para. Haastattelu). The questions are formatted beforehand and asked in a certain order, but the responses are open-ended (Ojasalo et al. 2015, 108).

4.6.1 Interview Implementation

A structured questionnaire interview was used as a method to gain additional points of view. The main goal of the interviews was to get context for building use case scenarios. The interview was performed for a person working as a full stack developer on a Ruby on Rails software, for a person working with error logging of an electronic health record system, and for a person working as a software developer and consultant. The interviews were conducted via email due to time constraints.

To get additional context for the use case scenarios, the team gathered information on three key areas: factors that weaken code quality, how outdated code manifests in software, and the kind of extra work erroneous code typically causes. To find out about the existing procedures and standards, the respondents were asked what role commenting on the code has in software maintenance, and whether there is a standardized way to comment on the code in the respondent's team. To gain more insights about tools and methods that already exist, the interview contained questions about the tools and logs used to monitor the code, the phases of fixing code afterwards,

and the AI tools the respondents use in their work. The interview questions can be found in Appendix 2.

4.6.2 Interview Results

The respondents identified rush and inadequate testing as the primary factors leading to weaker code quality. Secondary contributing factors included unclarity and elements that complicate code modification and bug fixing in further development. Such factors were typically caused by bad naming, inadequate typing, mixing different levels of abstraction in the same function or source code file, and the lack of conceptual integrity. Different trends and recommended practices were found to cause problems when applied to situations where they are not necessarily suitable.

Based on the interviews, outdated code manifests as slowness, crashes, or bugs. Furthermore, when free open-source libraries are used, especially those from a third party, it was found to be common that security updates and other support for old versions cease, requiring significant effort for necessary updates.

The respondents noted that erroneous code forces developers to first investigate the problem, followed by its remediation and re-testing. The correction of one error often causes new issues, leading to a chain reaction where each subsequent adjustment requires new tests and potential further work. Resolving errors in a production environment was cited as significantly more difficult and slower than in lower environments. Furthermore, a minor change during maintenance can require significant redesigning of existing code.

According to the respondents, in-code comments facilitate other employees' understanding of the code and their ability to modify it. It can be difficult to recall even one's own solutions; thus, adding comments can make future work easier. Furthermore, comments ease the transition for a new employee when another employee terminates their employment, enabling them to continue working on solutions more easily. Commenting was said to be important for documenting unconventional or non-obvious actions, but naming the functions and variables was mentioned to be more essential. There were no standardized ways of commenting on the code in the respondents' teams, and the practice was, in principle, kept minimal.

The tools that the respondents told they use currently in their work for monitoring existing code were Bitbucket, Bitbucket Pipelines, Heroku Papertrail, TrackJS, ErrorMailer, Git version control, and Jira. The logs that were mentioned in the interviews were error logs, access logs for HTTP traffic, and audit logs for business transactions.

The phases involved in resolving erroneous code reported by the respondents were investigating the problem and its affected areas, searching the code to find the root cause, and then correcting the bug and re-testing the correction. Depending on how complex the remediation is, it may require testing by peers before deployment to production. If the issue is reported by a client, they must be informed that the bug is resolved after the correction has been deployed. Depending on the case, sometimes a quick fix is necessary to handle the most acute situation, allowing time to plan a permanent way of remediating the code.

The AI tools that the respondents used in their work were ChatGPT, GitHub Copilot and Copilot for Microsoft 365 (Enterprise version based on GPT-4 architecture). It was specified that the version of GitHub Copilot in use is an enterprise version that does not use the input content to train its models.

4.7 Findings

Based on the knowledge gathered and the ideation process, the scope was refined to a Correlation AI Agent that monitors data and finds the root causes of technical debt, while keeping in mind that communication with humans is a key factor. The Correlation AI Agent could work on a very specific task in collaboration with humans, and in the future, even as part of a chain of AI Agents.

A key finding emerged regarding the required nature of the solution: autonomous agent functionality would significantly improve codebase maintenance and early-stage fault identification. This shift means that the maintenance initiative moves from being solely developer-driven to being initiated proactively by the Correlation AI Agent.

By mapping out the current challenges and researching competing solutions, the team determined the current problem, which is that other relevant context is not combined with code scanning in detecting, fixing, and preventing technical debt. Based on that, the research question was defined.

Benchmarking already existing solutions gave the team validation that there is currently no commonly known agentic AI solution that would consider all the factors of the research question. The team also got insights into the Correlation AI Agent's user interface. The majority of the AI tools that were benchmarked had a conversational user interface, which was regarded as a key point moving into the prototyping phase of the process.

The findings from the interviews indicate that rushed processes, unclear requirements, and inadequate testing weaken code quality. Early resolution of these issues and correction of code errors is crucial to prevent wasting resources on subsequent investigation, remediation, and retesting. While code comments arguably should be kept minimal, partly due to the lack of standardized methods for annotation, commenting the code may still aid future development, particularly for other developers and when implementing unconventional solutions. Significantly, the interviews revealed that no common AI tools are currently used for identifying technical debt or for scanning text among code or other project documents in relation to the code. The following chapter describes a detailed solution proposed in response to these findings.

5 The Human-in-the-Loop Code Repair and Maintenance Workflow with the Correlation AI Agent

The integration and governance of AI systems, particularly in software development, necessitate maintaining human involvement as a critical component. While advanced language models and coding assistants can analyze, modify, and document code autonomously, the human role remains essential for quality assurance and ethical decision-making.

This chapter presents a Customer Journey illustrating how a software developer and agentic AI can interactively collaborate to repair program code. The process follows the Human-in-the-Loop and Agent-in-the-Loop principles, emphasizing the cyclical cooperation between human expertise and artificial intelligence.

5.1 Process Description and Human-in-the-Loop Principles

The code repair and maintenance process is an iterative cycle where the Correlation AI Agent and the developer alternate actions until an acceptable correction level is

achieved. This dynamic structure closely resembles agile software development methodologies, where rapid feedback loops are essential for continuous improvement in both speed and quality (Manjunatha 2025). Successful outcomes throughout the entire cycle rely fundamentally on interaction, transparency, and iterative cooperation between the agent and the human expert.

The repair cycle can be initiated in two distinct ways, ensuring maximum flexibility in responding to emerging issues. First, it can begin proactively by the Correlation AI Agent. This automated process starts with the Correlation AI Agent continuously analyzing assigned data sources for anomalies, immediately triggering the rest of the workflow upon detection. Alternatively, the process can be triggered on demand by the developer. A human developer may directly instruct the Correlation AI Agent to analyze a specific code block, investigate a recent incident, or examine a particular system state at their discretion.

The core principle guiding this interaction is the combination of human supervision and powerful AI analytical capabilities, creating a robust Human-in-the-Loop framework (Shneiderman 2020). The entire correction process is considered complete only when the developer determines that the code has been sufficiently corrected and the underlying technical debt has been adequately addressed.

5.2 Mapping the Automated Correlation AI Agent Workflow

The primary objective of the Correlation AI Agent is to transform vast quantities of passive, multi-source observability data into active, actionable intelligence. This automated process precedes the first interaction with a human colleague. The Correlation AI Agent leverages existing IT investments (e.g., scanners, logging, and metric systems) to minimize overhead while focusing its analytical capabilities on root cause identification and solution proposal generation.

The Correlation AI Agent's operation is structured into two distinct phases: passive data intake utilizing existing tools and active analysis, which is the AI Agent's core function. This structured workflow ensures that all findings are fully contextualized with code history, system state, and proven best practices before being presented to a developer for the next step (Parry 2025). The following figure (Figure 2) illustrates the six independent work stages that constitute the full workflow.

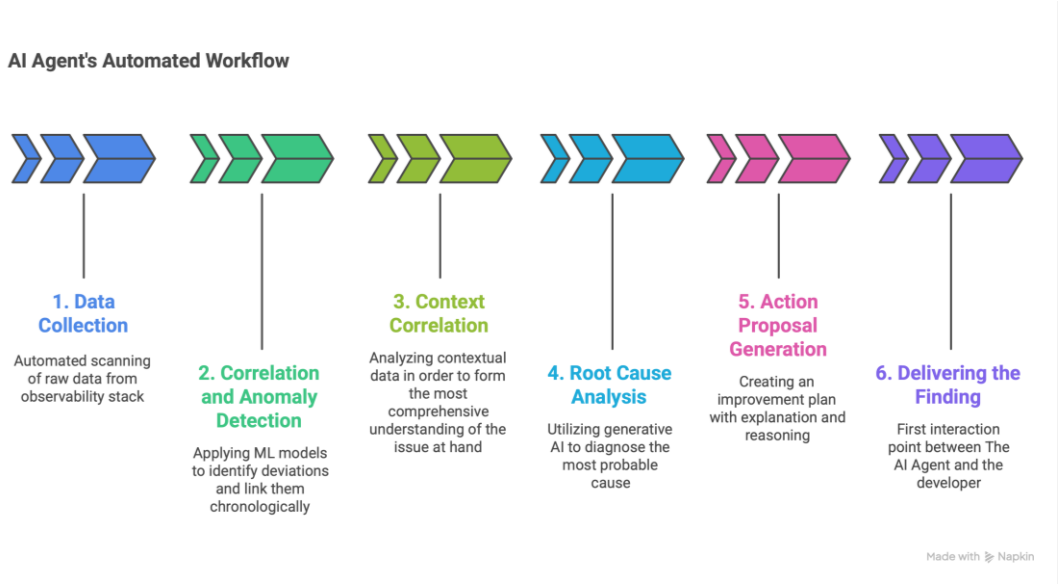


Figure 2: AI Agent's Automated Workflow

The Correlation AI Agent's autonomous process begins with six core stages designed to move from raw data ingestion to generating an actionable plan for a developer. First, the agent performs passive Data Collection and normalization. This involves continuously gathering raw data from the system's existing observability stack (logs, metrics, traces) and converting it into a unified, machine-readable format. Technologies supporting this stage include Fluentd, Prometheus, Jaeger, and a centralized Data Lake.

Following this, the Correlation AI Agent moves to Correlation and Anomaly Detection. This initial analytical step of the correlation process applies machine learning models to identify deviations and link them chronologically to specific system or code events. For instance, the system might note a spike in latency at 14:00 that is concurrent with the deployment of code block x. This is achieved primarily through rule-based ML models.

Subsequently, the agent executes Context Correlation. This critical step actively retrieves all necessary contextual data, including source code, Git history, related design documents, and static analysis reports, that are relevant to the correlated anomaly. Technologies utilized for detailed contextual mapping include Git history, SonarQube, generative AI, and Large Language Models (LLM).

Next, the agent performs Root Cause Analysis. It leverages generative AI capabilities to diagnose the single most probable cause of the detected issue, such as determining that

an unclear description in a requirements document led to a logical error in the production code. This sophisticated diagnostic work is powered by generative AI and LLMs.

The penultimate stage is Action Proposal Generation. Here, the Correlation AI Agent creates a detailed improvement plan, complete with reasoning based on the data sources used and concrete suggestions for code modification or process improvement.

Finally, the process concludes with Delivering the Finding. This stage represents the first interaction point with a human user, as the Correlation AI Agent presents the complete finding and action plan to a developer on its dedicated platform for human review and execution.

The technological and tool examples cited within this entire process are selected based on their status as industry-leading, domain-specific standards and open-source best practices required to fulfil each function. These selections reflect the necessity of grounding the workflow in robust, reliable infrastructure (Verma 2025).

5.3 The Developer Action Process

This part of the cycle is initiated immediately after the Correlation AI Agent presents its findings and action proposal (Stage 6) to the human colleague. The developer's overall workflow is segmented into three key categories: Receiving and Prioritizing, Validation and Planning, and concluding the cycle with Implementation and Feedback. The entire process requires, and is defined by, the critical integration of human judgment, expertise, and authorization to apply the proposed changes (Rahwan et al. 2019).

Three elements highlight the necessity of human involvement in this otherwise automated process. First, there is the issue of authorization and authority. Although the Correlation AI Agent conducts the analysis proactively and efficiently, it fundamentally lacks the authority to decide further steps or commit changes directly to production systems. The human colleague must therefore grant the necessary final authorization to proceed with any action. Second, context and experience remain invaluable. Despite gathering a vast array of technical and contextual data from appointed sources, the Correlation AI Agent lacks contextual understanding outside these defined data parameters. Consequently, human experience and domain knowledge are essential to confirm the diagnosis and cannot be entirely excluded from the decision-making loop. Finally, testing and validation of the final solution is the direct, non-delegable responsibility of the developer. In

summary, the Correlation AI Agent's ability to handle the repetitive, complex analytical steps significantly reduces the developer's workload, enabling them to focus their expertise on these critical validation and decision points as well as other high-value tasks.

5.3.1 Receiving and Prioritizing Phase

The developer's workflow segments sequentially follow the Correlation AI Agent's automated phase, beginning with the Receiving and Prioritizing stage, which defines the start of human intervention. The subsequent figure (Figure 3) visualizes this initial phase as part of the complete developer-Correlation AI Agent-cycle.

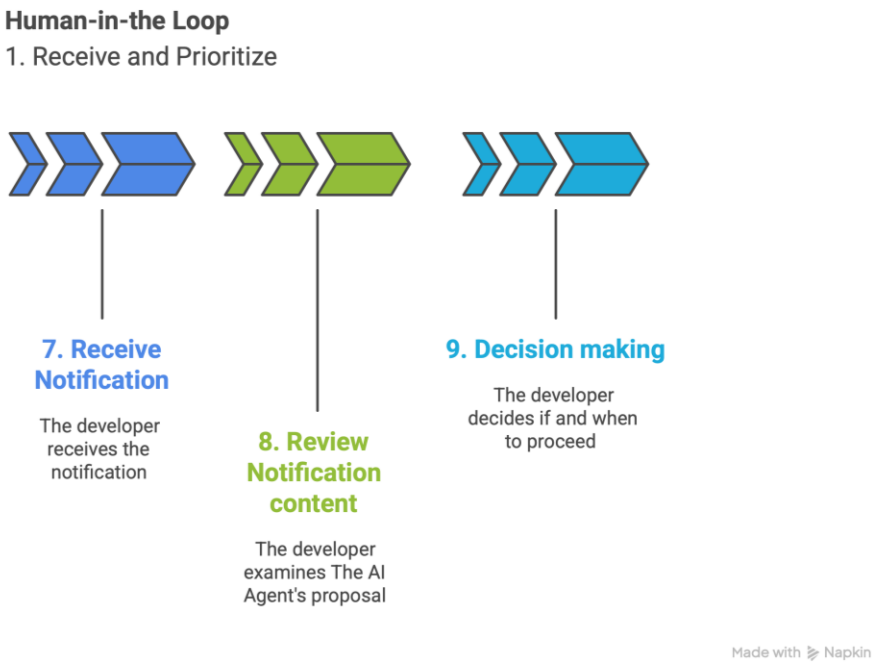


Figure 3: Human-in-the-Loop 1. Receive and Prioritize

The first step is Receiving Notification from the dedicated platform, which alerts the developer to the new findings and requires their attention. Subsequently, the developer proceeds to Reviewing the Notification: they meticulously examine the Correlation AI Agent's comprehensive report, which consists of the root cause analysis, correlation data, the specific code block in question, and the step-by-step action proposal.

Finally, this review culminates in Decision Making, where the developer assesses the Correlation AI Agent's detailed proposal and strategically prioritizes it in relation to other

tasks at hand, determining whether immediate execution or further investigation is necessary.

5.3.2 Validation and Planning Phase

Following the prioritization measures in the previous stage, the developer's next responsibility is to validate the Correlation AI Agent's root cause analysis and proposed solution, and to plan the implementation strategy. This stage emphasizes the human role as a critical filter, introducing contextual knowledge and experience-based judgment into the process before committing resources. The steps described below are visually presented in Figure 4.

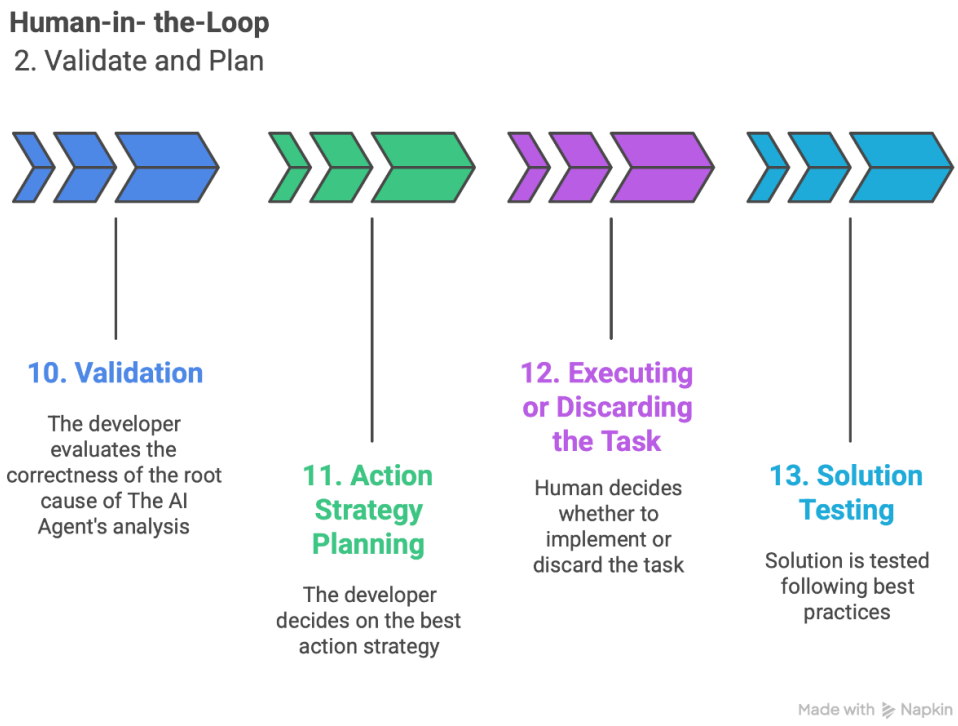


Figure 4: Human-in-the-Loop 2. Validate and Plan

The cycle commences with Validation. The developer evaluates the root cause analysis provided by the Correlation AI Agent, utilizing their experience to make the final assessment of whether the issue is truly caused by the suggested root cause. This critical evaluation ensures that the corrective action is directed at the correct problem.

This is immediately followed by Planning the Action Strategy, where the developer determines if the Correlation AI Agent's proposed action is the optimal solution. While the

Correlation AI Agent suggests *what* to do, the human decides *how* to implement it (e.g., opting for a thorough refactoring versus a quicker, tactical fix).

The developer then moves to Execution or Discarding the Task, making the final decision on whether and how to proceed with the action. The suggested improvement may be discarded due to external information unknown to the Correlation AI Agent (e.g., a planned architectural renewal in six months), which renders the fix unnecessary or inefficient.

Finally, the process concludes with Testing the Solution. The solution is rigorously tested before final approval, adhering to best practices established for code review and quality assurance (Parry, D. 2025).

5.3.3 Implementation and Interaction Phase

This final phase of the Developer Action Cycle, visually represented in Figure 5, focuses on closing the loop by implementing the validated solution, ensuring human quality assurance, deploying the fix, and providing crucial feedback to the Correlation AI Agent for continuous learning.

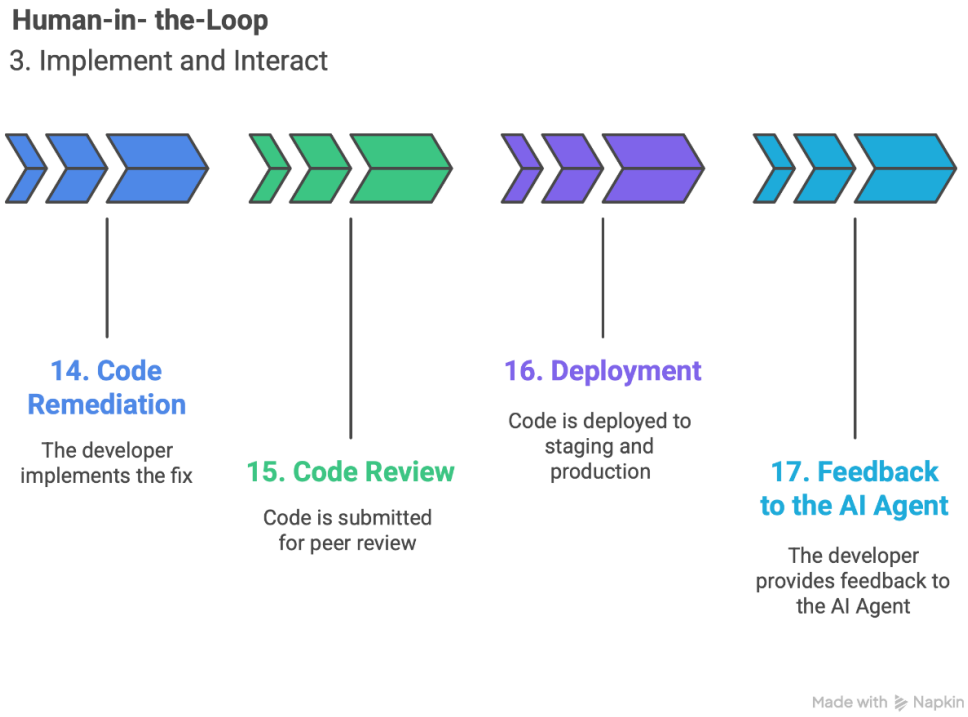


Figure 5: Human-in-the-Loop 3. Implement and Interact

The cycle continues with Code Remediation, where the developer implements the corrective actions identified in the planning stage, ensuring that all associated tests are successfully passed. Once the fix is implemented, the process moves to Code Review, where the code is submitted for peer inspection. This human quality assurance layer is vital and is not replaced by the Correlation AI Agent's recommendations.

After successful review, the code enters Deployment, passing through the CI/CD pipeline and being released sequentially to staging and finally to the production environment. The entire cycle concludes with Feedback to the Correlation AI Agent. The developer marks the issue as resolved and provides explicit feedback to its platform, such as: "Your diagnosis was X % correct, but the solution required Y." This feedback mechanism is essential as it refines the Correlation AI Agent's future learning models, ensuring continuous improvement in its diagnostic and proposal capabilities.

6 Prototype

A prototype is an early model of a product. The goal of creating a prototype is to simulate a finished product’s design and functionality, which enables testing concepts, gathering feedback, and improving design before developing the final product. (Interaction Design Foundation 2019; Knapp & Zeratsky 2025.) A prototype’s fidelity can vary from low to high with close resemblance to the final product (Interaction Design Foundation 2019).

6.1 Use Case Scenario

A use case scenario is a user story that defines the system’s process to accomplish a business goal. A system’s features and functional requirements can be tested by creating use cases and story-formed scenarios. They can also be used to identify potential exclusions and boundary constraints. (Hooda, Vandana, Yashwant, Sandeep & Manu 2023, 5.4.)

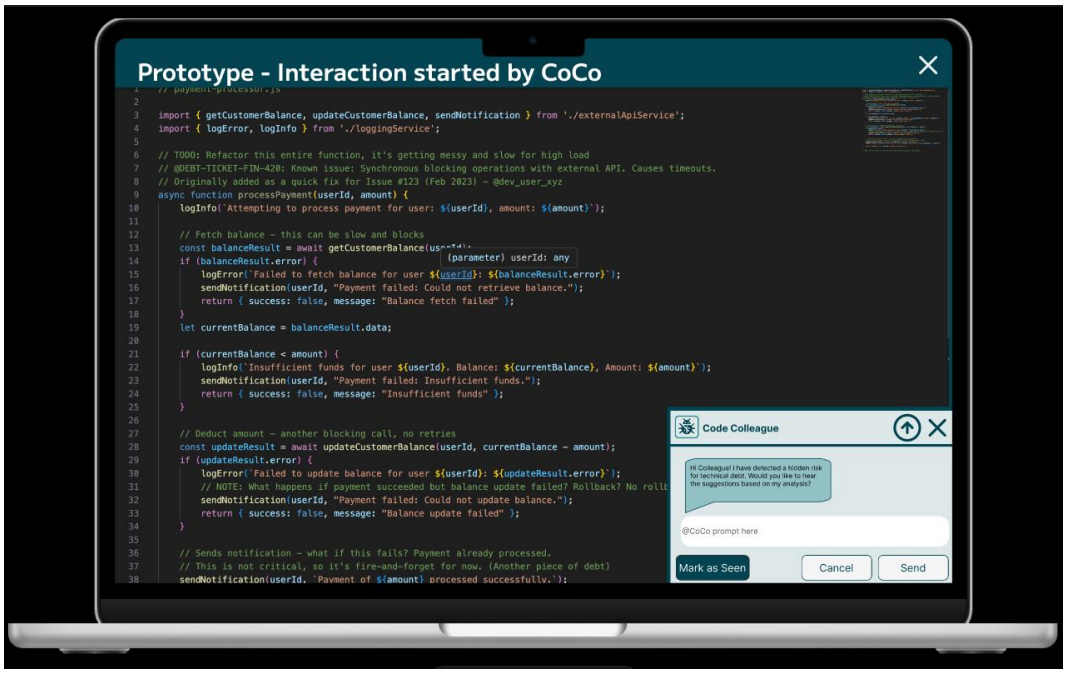
Based on the interview results, the use case scenario was defined around order management software, with the target user persona identified as a software developer responsible for code development and maintenance. The Correlation AI Agent finds a potential hidden risk for technical debt in the comments of the software’s code. The comments of the payment process code section are unclear and vague; therefore, the Correlation AI Agent suggests editing the comments to be more specific to avoid extra

investigation for the software's future developers. The Correlation AI Agent identifies an outdated section of code containing a logical error in the regular customer discount calculation. It points out a specific outdated integer that is causing some customers to receive overly generous discounts, which in turn causes direct financial loss for the company. The Correlation AI Agent then suggests correcting the integer based on the current business requirements.

6.2 Implementation of the Prototype

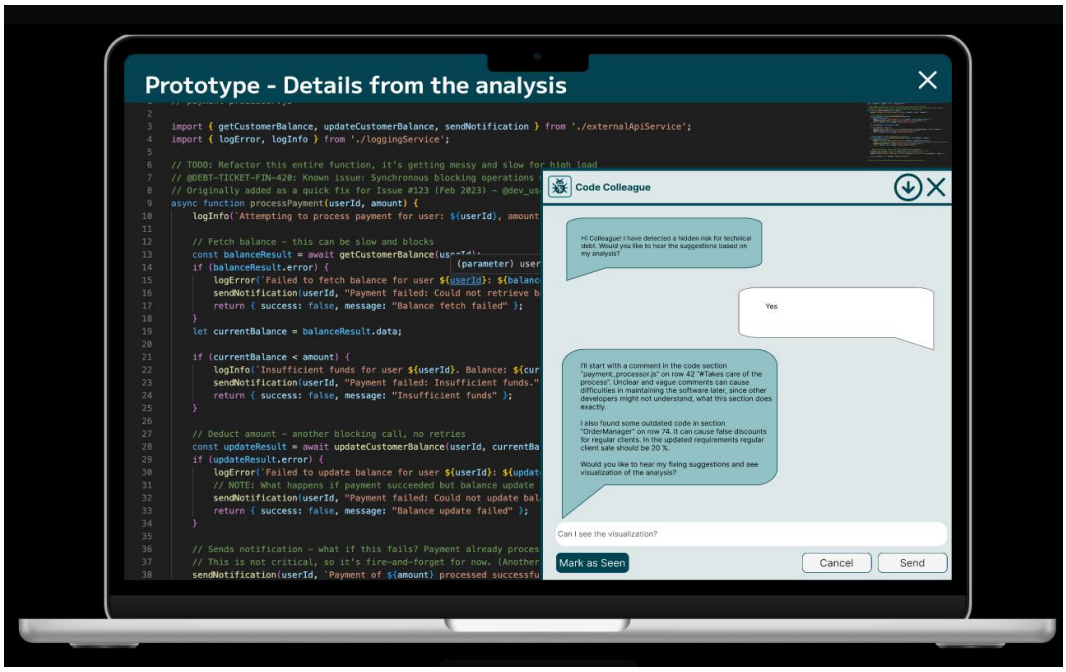
The prototyping process started by creating a prototyping plan and wireframes for the prototype in Miro. The prototype was created with Figma. The goal was to demonstrate the conversational user interface and a use case scenario of the Correlation AI Agent. It was presented as a laptop screen view with an image of a code editing program in the background to show how an actual use case scenario would look in the environment where the Correlation AI Agent could be used. The prototype was named Code Colleague, CoCo.

The Correlation AI Agent scans the code, in-code comments, commit messages, pull requests, and requirements of the software. When it detects potential hidden risks of technical debt, it starts interaction with the developer working on the software and requests whether suggestions based on its analysis are needed (Picture 1).



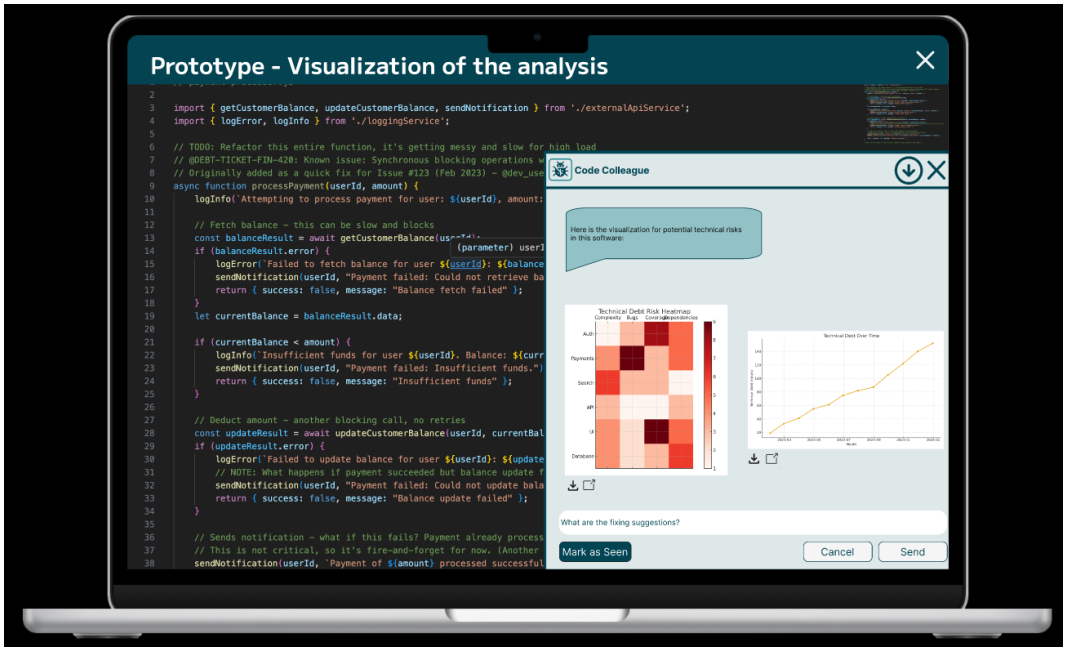
Picture 1: Prototype 1. Interaction Started by the Correlation AI Agent

The Correlation AI Agent can be prompted for necessary changes or additional information by writing in the chat. After the developer has confirmed that the suggestions based on the Correlation AI Agent's analysis are needed, further information about the potential risks of technical debt is offered (Picture 2).



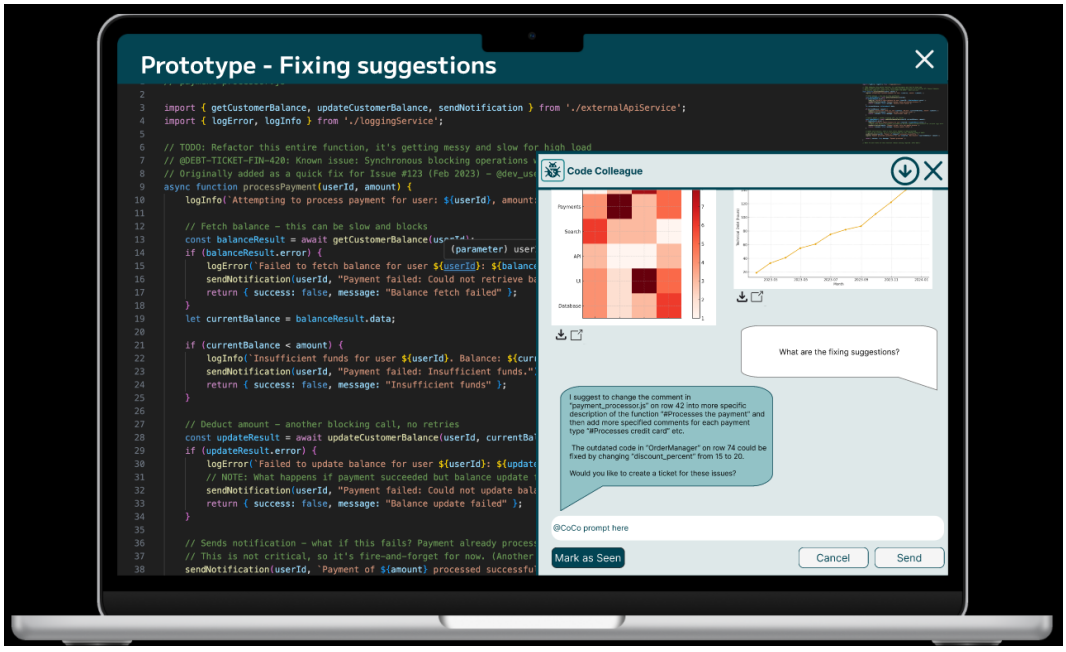
Picture 2: Prototype 2. Details from the Correlation AI Agent's Analysis

The Correlation AI Agent requests whether fixing suggestions or visualization of the analysis is needed. After the developer's request for visualization of the Correlation AI Agent's analysis, suitable figures are created to demonstrate the consequences of the technical debt of the issues found (Picture 3).



Picture 3: Prototype 3. Visualization of the Correlation AI Agent's Analysis

The figures can be expanded to a window and downloaded. Subsequently, the developer requests fixing suggestions for the issues. Clear suggestions are provided for each issue with specified information (Picture 4).



Picture 4: Prototype 4. Correlation AI Agent's Fixing Suggestions

Next, the Correlation AI Agent asks whether creating a ticket for the issue is necessary. The Correlation AI Agent operates accordingly until the potential risks of technical debt in the software are corrected or a human terminates the loop.

7 Conclusions, Future Scope and Development Directions

The objective of this project was to design and evaluate a solution capable of improving the efficiency of software maintenance processes, with a particular focus on identifying and reducing technical debt.

To address this question, a prototype of the Correlation AI Agent was developed. The solution aggregates technical code metrics with contextual information extracted from documentation, commit messages, and issue tracking systems. By correlating these heterogeneous data sources, the specified Correlation AI Agent can identify likely root causes of technical debt. This objective is achieved by ensuring a comprehensive problem analysis that incorporates all influencing data sources, regardless of their specific data type. A key feature of the proposed solution is the Correlation AI Agent's proactive role; where maintenance was previously developer-initiated, this task is now primarily driven by the Correlation AI Agent. This proactive approach is followed by continuous interaction as the Correlation AI Agent actively communicates with the user, poses clarifying questions, and updates its reasoning based on feedback. This interaction ensures that developers maintain a comprehensive understanding of the maintenance task, resulting in faster diagnosis and more informed decision-making.

The results indicate that such an approach can support more efficient maintenance workflows by reducing time spent on manual investigation, improving situational awareness, and enabling quicker correction of software faults. In practice, this may translate into reduced long-term technical debt, lower maintenance costs, and additional resources that can be redirected toward new development initiatives. The Correlation AI Agent represents a natural, realistic, and highly feasible next step for codebase maintenance, as it leverages existing tooling and infrastructure, thus protecting prior organizational investments.

Several directions for future development can be identified. First, the placement of the Correlation AI Agent within a larger AI Agent chain warrants further investigation. Defining

the structure and responsibilities of an AI Agent chain, as well as specifying the subsequent AI Agent in the workflow, would enable the creation of a coordinated multi-agent ecosystem. Such a system could allocate tasks intelligently across AI Agents with different capabilities, thereby improving scalability and automation potential.

Second, mechanisms for inter-agent collaboration should be formalized. This includes designing shared communication protocols, knowledge-transfer methods, and conflict-resolution strategies. A multi-agent-human-loop workflow, where agents collaborate with each other while continuously involving human experts for validation and oversight, represents a promising model. This hybrid approach combines the efficiency of autonomous computation with the contextual understanding and ethical judgment of human practitioners.

Third, a compelling framework for defining the Correlation AI Agent's integration is to view it as the creation of a defined “job role” within the development team. Just as human employees have defined positions and responsibilities, the Correlation AI Agent's inclusion necessitates a reorganization of tasks, with a specific work scope assigned to it. The human maintains oversight of the Correlation Agent's performance and retains the ultimate authority over task execution. This strategic supervision ensures that it develops into a highly autonomous system, thereby continuously automating routine tasks and reducing the developer's intervention time. Future work should also involve defining an auditing system for the Correlation AI Agent, such as an algorithm audit or equivalent protocols corresponding to human performance monitoring.

In summary, the implemented Correlation AI Agent demonstrates the potential to enhance the diagnosis of technical debt and streamline software maintenance processes. Continued development toward multi-agent orchestration and human-inclusive workflows may further advance its applicability in complex and large-scale software environments.

References

Acharya, D., Divya, B. & Kuppan, K. 2025. Agentic AI: Autonomous Intelligence for Complex Goals—A Comprehensive Survey. IEEE Xplore. Accessed 13 October 2025. https://ieeexplore.ieee.org/abstract/document/10849561

Allman, E. 2012. Managing technical debt. Communications of the ACM. Volume 55. Issue 5. Pages 50-55. https://dl.acm.org/doi/epdf/10.1145/2160718.2160733

Battacharjee, S. 2025. Static vs. dynamic code analysis: A comprehensive guide. Accessed 14 October 2025. https://vfunction.com/blog/static-vs-dynamic-code-analysis/

Chomal, V., Saini, J. 2014. International Journal of Engineering, Innovation & Research. Volume 3. Issue 4. Pages 409-416. https://www.researchgate.net/profile/Jatinderkumar-Saini/publication/281965276_Significance_of_Software_Documentation_in_Software_Development_Process/links/55ffdc6008aeba1d9f841187/Significance-of-Software-Documentation-in-Software-Development-Process.pdf

CodeScene. 2025a. Fix and Prevent Tech Debt inside your IDE. Accessed 19 October 2025. https://codescene.com/product/integrations/ide-extensions

CodeScene 2025b. Manage Technical Debt to Maximize Developer Productivity. Accessed 19 October 2025. https://codescene.com/

CodeScene 2025c. Prioritize and manage technical debt. Accessed 19 October 2025. https://codescene.com/use-cases/technical-debt-management

Cunningham, W. 1992. Introduction to the Technical Debt Concept. Agile Alliance. Accessed 11 October 2025. https://www.agilealliance.org/wp-content/uploads/2016/05/IntroductiontotheTechnicalDebtConcept-V-02.pdf

Downie, A. & Finn, T. 2025. Agentic AI vs. generative AI. IBM. Accessed 16 October 2025. https://www.ibm.com/think/topics/agentic-ai-vs-generative-ai

Franzese, M. & Antonella, I. 2018. Correlation Analysis. Institute of Applied Mathematics “Mauro Picone”. Accessed 31 October 2025. https://www.sciencedirect.com/science/article/abs/pii/B9780128096338203580?via%3Dihub

Friedland B., De Santis M. 2025. Thoughtworks. Accessed 26 October 2025. https://www.thoughtworks.com/content/dam/thoughtworks/documents/whitepaper/tw_whitepaper_dont_waste_a_tech_debt_crisis.pdf

Garg, V. 2025. Designing the Mind: How Agentic Frameworks Are Shaping the Future of AI Behavior. Journal of Computer Science and Technology Studies, 7(5), 182-193. https://doi.org/10.32996/jcsts.2025.7.5.24

GeeksforGeks. 2025. What is SonarQube. Accessed 20 October 2025. https://www.geeksforgeeks.org/devops/sonarqube/

Glikson, E., & Woolley, A. W. 2020. Human trust in artificial intelligence: Review of empirical research. Academy of Management Annals. Accessed 19 October 2025. https://journals.aom.org/doi/10.5465/annals.2018.0057

Greptile. No date. User Manual. Custom Context & Learning. Accessed 22 October 2025. https://www.greptile.com/docs/code-review-bot/custom-context

Greptile. 2025. The AI Code Reviewer. Accessed 20 October 2025. https://www.greptile.com/

Gupta D. 2024. Introducing Greptile 2.0. Greptile. Accessed 20 October 2025. https://www.greptile.com/blog/greptile-2

Hakala, J. 2024. Laadullisen tutkimuksen ABC. Menetelmäopas opinnäytteen tekijälle. E-book. Helsinki: Gaudeamus.

Hooda, S., Vandana, M. S., Yashwant, S., Sandeep, D. & Manu, S. 2023. Agile Software Development. Trends, Challenges, and Applications. E-Book. Hoboken: Wiley-Scrivener.

Interaction Design Foundation. 2019. What are Prototypes? Accessed 31 October 2025. https://www.interaction-design.org/literature/topics/prototypes

Kestilä-Kekkonen, E. 2021. Kvantitatiivisen tutkimuksen verkkokäsikirja. Kovarianssi ja Korrelaatio. Tampere: Yhteiskuntatieteellinen tietoarkisto. Accessed 20 October 2025. https://www.fsd.tuni.fi/fi/palvelut/menetelmaopetus/kvanti/korrelaatio/korrelaatio/

Knapp, J. & Zeratsky, J. 2025. The Design Sprint. Accessed 7 October 2025. https://www.thesprintbook.com/the-design-sprint

Knapp, J., Zeratsky, J. & Kowitz, B. 2016. Sprint. Solve big problems and test new ideas in just five days. E-book. New York: Simon & Schuster.

Kumar, S. 2023. Data Silos -A Roadblock for AIOps. https://www.researchgate.net/publication/334276277_Bridging_Data_Silos_Using_Big_Data_Integration

Laurea-ammattikorkeakoulu. 2022. Nokia as key partner for Laurea. Accessed 8 October 2025. https://www.laurea.fi/en/current-topics/news/nokia-as-key-partner-for-laurea/

Manjunatha, R. (2025). Building Autonomous AI systems with Looping Agents from Google's Agent Development Kit(ADK). Accessed 19 October 2025. https://medium.com/google-cloud/building-autonomous-ai-systems-with-looping-agents-from-googles-agent-development-kit-adk-cb9d0d4997a5

Mucci, T. 2025. What is technical debt? Accessed 13 October 2025. https://www.ibm.com/think/topics/technical-debt

Nokia 2025a. Nokia Corporation. Accessed 8 October 2025. https://www.nokia.com/people/nokia-corporation

Nokia 2025b. We are Nokia. Accessed 23 October 2025. https://www.nokia.com/we-are-nokia/

Nokia 2025c. Nokia adds new Agentic-AI capabilities across its autonomous networks portfolio #MWC25. Accessed 21 November 2025. https://www.nokia.com/newsroom/nokia-adds-new-agentic-ai-capabilities-across-its-autonomous-networks-portfolio-mwc25/

Ojasalo, K., Moilanen, T. & Ritalahti, J. 2015. Kehittämistyön menetelmät. Uudenlaista osaamista liiketoimintaan. E-book. Helsinki: Sanoma Pro Oy.

Parry, D. 2025. Top 7 Code Review Best Practices in 2025. Qodo. Accessed 18 October 2025. https://www.qodo.ai/blog/code-review-best-practices/

Powerdrill. No date. Introduction. Accessed 23 October 2025. https://docs.powerdrill.ai/get-started/introduction

Protalinski, E. 2023. Accessed 24 October 2025. https://www.honeycomb.io/blog/what-is-observability-key-components-best-practices

QQ. 2025. How to Calculate Correlation Coefficient with AI | Powerdrill. Powerdrill. Accessed 23 October 2025. https://powerdrill.ai/blog/correlation-coefficient

Raghuvanshi, K., Scori, E. & Grover, M. 2024. Can AI solve the rising costs of technical debt? Accessed 16 October 2025. https://www.alixpartners.com/insights/102jlar/can-ai-solve-the-rising-costs-of-technical-debt/

Rahwan, I., Cebrian, M., Obradovich, N., Bongard, J., Bonnefon, J.-F., Breazeal, C., ... Wellman, M. (2019). Machine behaviour. Nature. Accessed 19 October 2025. https://www.nature.com/articles/s41586-019-1138-y

Sawant, P. 2025. Agentic AI: A Quantitative Analysis of Performance and Applications. Read on 8.10.2025. https://www.preprints.org/frontend/manuscript/b540908df60641a985f056de30899adb/download_pub

Shneiderman, B. 2020. Human-centered artificial intelligence: Reliable, safe & trustworthy. International Journal of Human–Computer Interaction. Accessed 19 October 2025. https://www.tandfonline.com/doi/full/10.1080/10447318.2020.1741118?scroll=top&needAccess=true

Sonar. 2025a. SonarQube. Start with secure, high quality code built for adaptability. Accessed 20 October 2025. https://www.sonarsource.com/sem/products/sonarqube/

Sonar. 2025b. SonarQube ide. Extend SonarQube into your IDE for integrated code quality and security. Accessed 20 October 2025. https://www.sonarsource.com/products/sonarlint/

Sonar. 2025c. SonarQube Server. Advanced linter for better code quality and stronger security. Accessed 20 October 2025. https://www.sonarsource.com/products/sonarqube/server/

Tsidulko, J. 2023. Accessed 15 October 2025. https://www.oracle.com/sa/database/data-silos/#:~:text=Data%20silos%20are%20repositories%20of,%2C%20missing%2C%20or%20incomplete%20data.

Verma, H. 2025. 15 Open source Tools to Enhance Software Reliability. Intelligent HQ. Accessed 18 October 2025. https://www.intelligenthq.com/15-open-source-tools-to-enhance-software-reliability/

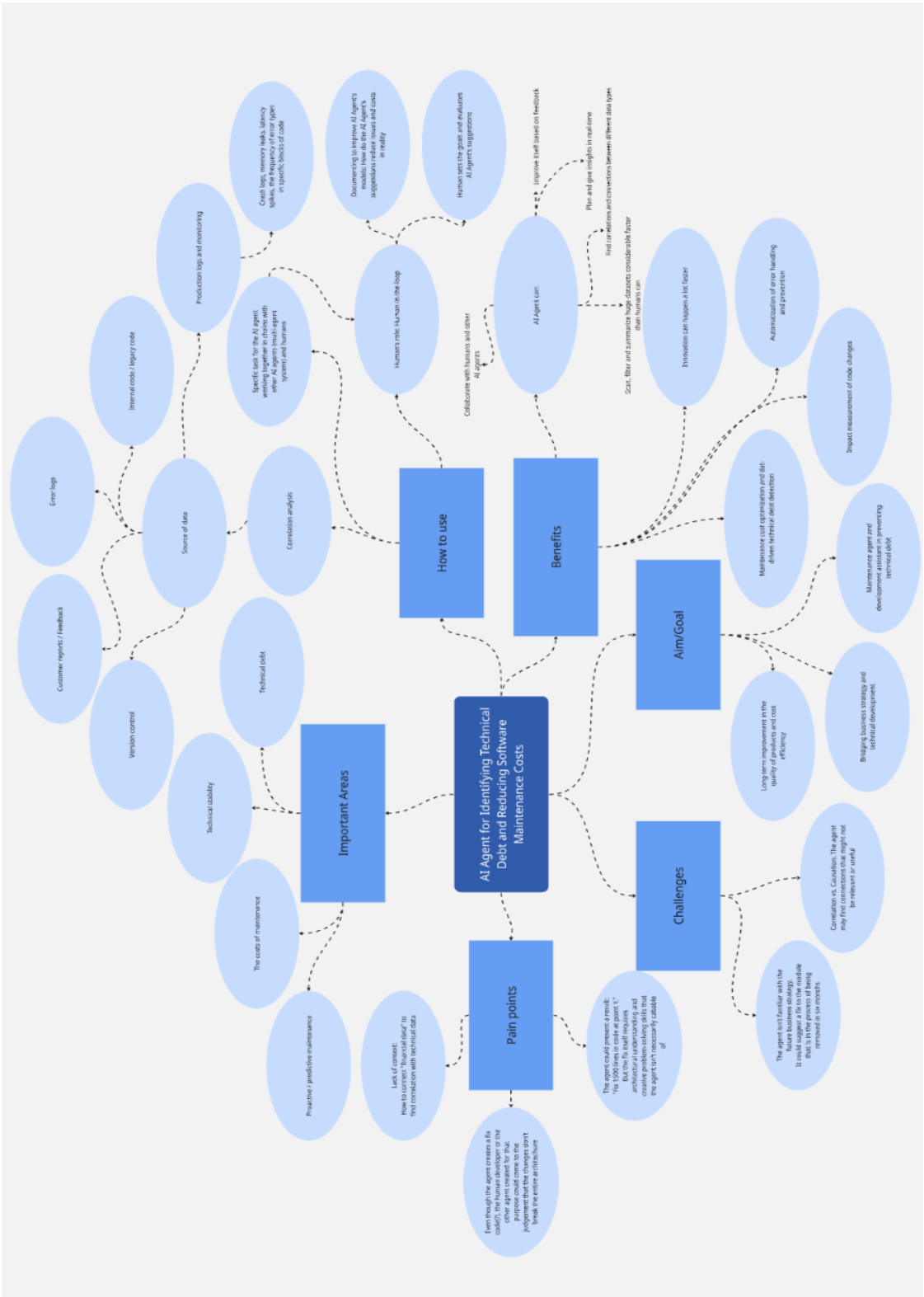
Figures

Pictures

Tables

Appendices

Appendix 1: Mind Map of the Most Important Findings



Appendix 2: Structured Questionnaire Interview Questions

We are conducting a Design Sprint thesis project as part of our studies at Laurea University of Applied Sciences. The subject of the project is the expense of technical debt in software maintenance. As part of the thesis, we are gathering research material in response to the questions below. The answers are used anonymously as material for our thesis.

You can respond to the questions applicable to your work in free form. Due to the strict schedule of Design Sprint implementation, we hope to get answers as quickly as possible.

What work tasks do you have regarding software development?

What factors weaken the quality of code?

How do outdated code solutions manifest in software?

What kind of extra work does erroneous code cause?

What role does commenting on the code have in software maintenance?

Is there a standardized way of commenting on the code in your team?

What tools and log files are used to monitor the existing code?

What are the phases of fixing code afterwards?

What AI tools do you use in your work?